

# CE2702 Scope — Implementation Guide

PO Notes New Persistency — 7 Activities with Why and How

EKKT\_TEXT | Handler Class | TTXOB Registration | SDM | OData V4 POC

SAP S/4HANA Procurement | EKKO / EKPO | 2026-06-30

| # | Activity                              | Why Required                                                       | Deliverable                             |
|---|---------------------------------------|--------------------------------------------------------------------|-----------------------------------------|
| 1 | Clarify Notes Reuse Compatibility     | Entire architecture decision depends on this — blocks table design | Architecture Decision Record signed off |
| 2 | Design EKKT_TEXT Table                | Handler class cannot be coded without table structure              | EKKT_TEXT Active in SE11                |
| 3 | Implement ZCL_PO_RSTXT_PERSISTENCE    | Without it STXH/STXL always used — new table never populated       | Handler class Active in SE24            |
| 4 | Register in TTXOB for EKKO/EKPO       | Without TTXOB registration handler class is never called           | EKKO+EKPO routing active                |
| 5 | Add SDM_VERSION to EKKO               | SDM framework needs status field to track migration progress       | SDM_VERSION field in EKKO via Append    |
| 6 | Implement ZCL_SDM_EKKO_TEXT_MIGRATION | Historic texts in STXH never migrated without SDM class            | SDM class Active — ready to run         |
| 7 | POC dual-write via OData V4 API       | Proves 2708 delivery works before committing full effort           | POC proven in DDCI system               |

## Dependency Chain

Activity 1 (Clarify Notes Reuse) — decision made



Activity 2 (Design EKKT\_TEXT) — table exists in SE11



Activity 3 (Handler Class)

■ class active



Activity 5 (SDM\_VERSION to EKKO)

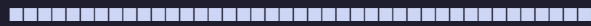
■ field in EKKO



Activity 4 (TTXOB Registration)

Activity 6 (SDM Class)

■ routing active                      ■ SDM class ready



Activity 7 (POC in DDCI)



POC results → 2708 full implementation

# 1

## Clarify Notes Reuse Compatibility

*The single most critical question — blocks all other activities*

This is a BLOCKER. Until this is answered, Activities 2-7 could go in the wrong direction. If Notes Reuse RAP only supports generic table SGBT\_NTCEONT, then EKKT\_TEXT design changes completely.

### What Must Be Clarified

| Open Question                                                              | Why It Blocks                                                           |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Can Notes Reuse RAP consume EKKT_TEXT (own table) instead of SGBT_NTCEONT? | If NO → must use SGBT_NTCEONT → redesign entire table approach          |
| Has Notes Reuse RAP improved since Sales team evaluated and rejected it?   | If YES → generic table may now be viable → avoid building own table     |
| Is there a supported way to register own table with Notes Reuse RAP?       | If YES → defines EKKT_TEXT structure and handler interface requirements |

### Step-by-Step — How to Clarify

#### 1 Schedule sync with Notes Reuse RAP team

Contact: Notes Reuse RAP team (contact TBD from CE2702/CE2708 meeting) Bring: Sales SDD design doc, EKKT\_TEXT draft design, open questions list Ask: Can own table be registered? What interface is needed?

#### 2 Schedule sync with SAP Script X team

Ask: Is their own-table migration guideline compatible with Notes Reuse RAP? Ask: What interface does the handler class need to expose for Notes Reuse?

#### 3 Schedule sync with Sales team (Chris, Karen Fang)

Ask: Did they evaluate Notes Reuse RAP? Why was it rejected? Ask: Can SDTP\_TEXT now be used with Notes Reuse? What changed?

#### 4 Document the Architecture Decision

Write Architecture Decision Record (ADR): Chosen option: EKKT\_TEXT (own table) OR SGBT\_NTCEONT (generic) Reasoning and trade-offs Sign-off: Nils Hartmann + Rajesh + architects

■ Deliverable: Architecture Decision Record signed off by PO product team.

## 2

## Design EKKT\_TEXT Table

Application-specific PO text table — must be designed before coding handler

Why better than STXH: TEXT\_CONTENT = plain SQL readable string. EBELN+EBELP = typed keys (not generic CHAR70). Auth fields stored directly. CDS views can be built on top.

## Proposed EKKT\_TEXT Field Structure

| Field         | Type | Length | Key | Description                           |
|---------------|------|--------|-----|---------------------------------------|
| MANDT         | CLNT | 3      | ✓   | Client                                |
| EBELN         | CHAR | 10     | ✓   | PO Number — typed, NOT generic CHAR70 |
| EBELP         | NUMC | 5      | ✓   | Item — 00000 = header texts           |
| TEXT_OBJECT   | CHAR | 10     | ✓   | EKKO (header) or EKPO (item)          |
| TEXT_ID       | CHAR | 4      | ✓   | F01, F02, F04 etc.                    |
| LANGUAGE      | LANG | 1      | ✓   | Language key                          |
| TEXT_CONTENT  | STRG | —      |     | Plain text string — ITF converted     |
| REF_TEXT_OBJ  | CHAR | 10     |     | Reference source object               |
| REF_TEXT_ID   | CHAR | 4      |     | Reference source text ID              |
| REF_TEXT_NAME | CHAR | 70     |     | Reference source document ID          |
| CREATED_BY    | CHAR | 12     |     | Created by (sy-uname)                 |
| CREATED_AT    | DATS | 8      |     | Created date                          |
| CHANGED_BY    | CHAR | 12     |     | Changed by                            |
| CHANGED_AT    | DATS | 8      |     | Changed date                          |
| AUTH_EKORG    | CHAR | 4      |     | Purchase Organization — for auth/DCL  |
| AUTH_BUKRS    | CHAR | 4      |     | Company Code — for auth/DCL           |
| AUTH_BSART    | CHAR | 4      |     | Document Type (NB, UB etc.)           |

## Step-by-Step — How to Create in SE11

## 1 Review and agree table structure with architect

Present EKKT\_TEXT design to Nils Hartmann / Rajesh Confirm: field names, auth fields, TEXT\_CONTENT as STRG vs CHAR132 rows Get sign-off before creating in SE11

## 2 SE11 → Database Table → EKKT\_TEXT → Create

Short Description: PO New Text Persistency Table Delivery Class: A Data Browser: Display/Maintenance Allowed Enhancement Category: Can Be Enhanced (Deep)

## 3 Add all fields from the table above

Key fields first (MANDT through LANGUAGE) Then non-key fields (TEXT\_CONTENT through AUTH\_BSART) Use standard data elements where possible (EBELN, EBELP, TDSPRAS etc.)

## 4 Set Technical Settings

Data Class: APPL0 (Application transaction data) Size Category: 2 (10,000–100,000 rows — will grow with PO volume) Buffering: Not Allowed (frequently written)

---

5

**Save → Activate (Ctrl+F3)**

Status must show: Active Package: ZMMPOC (or agreed package) Transport: assign to CE2702 workbench transport

---

6

**Create secondary index**

SE11 → EKKT\_TEXT → Indexes → Create → ZPO\_TXT\_I01 Fields: EBELN, TEXT\_OBJECT, TEXT\_ID Activate index

---

---

■ Deliverable: EKKT\_TEXT Active in SE11 with all fields and index.

---

## 3

## Implement ZCL\_PO\_RSTXT\_PERSISTENCE Handler Class

Inherits CL\_RSTXT\_PERSISTENCE\_FRAMEWORK — 5 methods + IS\_MIGRATION\_FINISHED

This is the automatic routing mechanism. Once registered in TTXOB, every SAVE\_TEXT/READ\_TEXT call for EKKO/EKPO goes through this class. No changes needed in ME21N/ME22N — the framework handles routing transparently.

## Class Definition

Class name: ZCL\_PO\_RSTXT\_PERSISTENCE

Inherits from: CL\_RSTXT\_PERSISTENCE\_FRAMEWORK

Final: Yes

Instantiation: Public

Description: PO Notes New Persistency Handler

## 5 Methods to Redefine + IS\_MIGRATION\_FINISHED

| Method                  | Triggered By            | Logic Summary                                                                                                                     |
|-------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| CREATE                  | SAVE_TEXT — new text    | Parse EBELN+EBELP from TDNAME. Check TDREF flag. Convert ITF→string. INSERT EKKT_TEXT. If SDM not done → also write STXH.         |
| CHANGE                  | SAVE_TEXT — update text | Same as CREATE but UPDATE. If user modified a reference → CLEAR REF fields permanently.                                           |
| DELETE                  | DELETE_TEXT             | DELETE from EKKT_TEXT. If SDM not done → also delete from STXH.                                                                   |
| READ                    | READ_TEXT               | If IS_MIGRATION_FINISHED=FALSE → read STXH/STXL. If TRUE → read EKKT_TEXT. Convert TEXT_CONTENT string→ITF for return.            |
| READ_HEADERS_VIA_RANGES | SELECT_TEXT             | If IS_MIGRATION_FINISHED=FALSE → read STXH headers. If TRUE → read EKKT_TEXT headers.                                             |
| IS_MIGRATION_FINISHED   | Called internally       | CL_SDM_PROC_STATUS_API=>IS_SDM_FINISHED for ZCL_SDM_EKKO_TEXT_MIGRATION. Result buffered for dialog session — no repeated SELECT. |

## Key TDNAME Parsing for This System

```
" EKKO (header): TDNAME = PO number only

IF im_header-tdobject = 'EKKO'.

    lv_ebeln = im_header-tdname.    " e.g. '4500077744'

    lv_ebelp = '00000'.

ELSE.    " EKPO (item): NO SPACE — confirmed for this system

    lv_ebeln = im_header-tdname(10).    " '4500077744'

    lv_ebelp = im_header-tdname+10(5).    " '00010' — offset 10, no space

ENDIF.
```

## 1

## SE24 → ZCL\_PO\_RSTXT\_PERSISTENCE → Create

Class type: Regular Class. Inherits: CL\_RSTXT\_PERSISTENCE\_FRAMEWORK. Final: Yes.

2

**Redefine IS\_MIGRATION\_FINISHED first**

METHOD is\_migration\_finished. rv\_finished = cl\_sdm\_proc\_status\_api=>is\_sdm\_finished( i\_sdm\_name = 'ZCL\_SDM\_EKKO\_TEXT\_MIGRATION' ). ENDMETHOD. This controls all read/write routing — implement before other methods.

3

**Implement CREATE method**

Parse TDNAME → EBELN+EBELP. Check TDREF='X' (reference) vs ' ' (copy). Call CONVERT\_ITF\_TO\_STREAM\_TEXT for copy. INSERT EKKT\_TEXT. If IS\_MIGRATION\_FINISHED=FALSE → also call standard STXH write.

4

**Implement CHANGE method**

Same as CREATE but UPDATE EKKT\_TEXT. If content written and REF fields were populated → CLEAR REF fields (reference broken).

5

**Implement DELETE method**

DELETE FROM EKKT\_TEXT by key. If IS\_MIGRATION\_FINISHED=FALSE → also delete from STXH.

6

**Implement READ method**

If IS\_MIGRATION\_FINISHED=FALSE → call READ\_TEXT to get from STXH/STXL. If TRUE → SELECT from EKKT\_TEXT → CONVERT\_STREAM\_TO\_ITF\_TEXT → return ITF table.

7

**Implement READ\_HEADERS\_VIA\_RANGES**

Route to STXH or EKKT\_TEXT based on IS\_MIGRATION\_FINISHED. Return text headers in expected THEAD format.

8

**Activate all methods (Ctrl+F3)**

All 6 methods must be blue (active) before TTXOB registration.

■ Deliverable: ZCL\_PO\_RSTXT\_PERSISTENCE Active with all 6 methods.

## Register in TTXOB for EKKO and EKPO

### 4

*Without TTXOB registration the handler class is NEVER called*

■ This is the activation step. Once TTXOB is updated, ALL applications using SAVE\_TEXT/READ\_TEXT for EKKO/EKPO will route through ZCL\_PO\_RSTXT\_PERSISTENCE. Test in sandbox first — do NOT register in production until POC is verified.

#### 1 SE11 → Table TTXOB → Change mode

TTXOB is the text object definition table. Alternative: SPRO → MM → Purchasing → Purchase Order → Texts for Purchase Order → Define Text Types for Purchase Order Header → check HANDLER\_CLASS field

#### 2 Find row: TDOBJECT = 'EKKO'

Set field: HANDLER\_CLASS = 'ZCL\_PO\_RSTXT\_PERSISTENCE' Save → assign transport

#### 3 Find row: TDOBJECT = 'EKPO'

Set field: HANDLER\_CLASS = 'ZCL\_PO\_RSTXT\_PERSISTENCE' Save → assign transport

#### 4 Verify registration

SE16N → TTXOB → filter TDOBJECT = EKKO → confirm HANDLER\_CLASS populated SE16N → TTXOB → filter TDOBJECT = EKPO → confirm HANDLER\_CLASS populated

#### 5 Test dual-write immediately in sandbox

ME21N → create PO with header text → Save Check 1: SE16N → EKKT\_TEXT → row exists with TEXT\_CONTENT ✓ (new table) Check 2: SE16N → STXH → row exists for EKKO (old table) ✓ (IS\_MIGRATION\_FINISHED=FALSE) Both must be written — dual-write confirmed

#### 6 Test READ routing

ME23N → open same PO → Header → Texts → text still visible ✓ IS\_MIGRATION\_FINISHED=FALSE → reads from STXH → legacy apps unchanged

■ Deliverable: EKKO and EKPO routing active. Dual-write confirmed in sandbox.



## 5

## Add SDM\_VERSION Field to EKKO

*SDM framework tracks migration progress via this field in root table*

WHY: SDM class selects packages of unmigrated POs using WHERE sdm\_version < target. IS\_MIGRATION\_FINISHED checks if ALL EKKO rows have sdm\_version = target. Without this field the SDM class cannot track which POs are migrated.

## 1 Coordinate with PO standard team (Nils/Jiss)

EKKO may already have SDM\_VERSION from another SDM (e.g. SDSLS\_SOFI clash in Sales) Agree on: field name, version constant value, numbering sequence Ensure no clash with other active SDMs on EKKO

## 2 Create Append Structure ZEKKO\_TEXT\_SDM

SE11 → Structure → ZEKKO\_TEXT\_SDM → Create Add field: SDM\_VERSION TYPE NUMC LENGTH 2 Short Description: SDM Version — PO Text Migration Activate structure

## 3 Assign Append Structure to EKKO

SE11 → EKKO → Change → Extras → Append Structure Enter: ZEKKO\_TEXT\_SDM → Activate NOTE: Direct EKKO modification not allowed in public cloud — must use Append

## 4 Set SDM\_VERSION in PO save processing

When NEW PO created: set SDM\_VERSION = target version constant When PO CHANGED: If SDM\_VERSION < target → keep old (not yet migrated by SDM) If SDM\_VERSION = target → write to EKKT\_TEXT only (migration complete for this PO)

## 5 Define version constant

Create constant in class or interface: IF\_ZPO\_SDM\_CONSTANTS=>PO\_TEXT\_SDM\_VERSION = '03' (or agreed value) Used in: SDM class GET\_STATUS\_VALUE\_DONE + handler IS\_MIGRATION\_FINISHED

## 6 Verify

ME21N → create new PO → SE16N → EKKO → confirm SDM\_VERSION field populated Value = target version constant ✓

■ Deliverable: SDM\_VERSION field in EKKO via Append Structure. Version constant defined.

## 6

## Implement ZCL\_SDM\_EKKO\_TEXT\_MIGRATION

Background batch job — migrates ALL historic STXH/STXL texts to EKKO\_TEXT

WHY: The handler class (Activity 3) handles NEW texts from registration date onwards. But millions of EXISTING PO texts are only in STXH/STXL and will never trigger SAVE\_TEXT again. This SDM class migrates all historic data in background — without it IS\_MIGRATION\_FINISHED can never become TRUE.

## SDM Methods to Implement

| Method                | Returns              | Purpose                              |
|-----------------------|----------------------|--------------------------------------|
| GET_TABLE_NAME        | 'EKKO'               | Root table for packaging             |
| GET_CLIENT_FIELD      | 'MANDT'              | Client field name                    |
| GET_SELECTIVE_FIELD   | 'EBELN'              | Key field for packages               |
| GET_STATUS_FIELD      | 'SDM_VERSION'        | Migration status field               |
| GET_STATUS_VALUE_DONE | target version const | Value meaning migration done         |
| GET_PACKAGE_SIZE      | 30000                | POs per package — ~1 min per package |
| MUST_RUN              | abap_true            | Always run this SDM                  |
| MIGRATE_DATA          | —                    | Main migration logic (see below)     |
| MIGRATE_DATA_FINISHED | —                    | Check if all EKKO rows migrated      |

## MIGRATE\_DATA — Logic Summary

MIGRATE\_DATA logic for each package of EKKO rows:

Step 1: Select package of unmigrated POs

```
SELECT ebeln, ekorg, buksr, bsart FROM ekko

WHERE sdm_version < target_version (using framework where condition)

UP TO 30000 ROWS.
```

Step 2: For each PO — read all text headers from STXH

```
Header: STXH WHERE tdoobject='EKKO' AND tdname = ebeln

Items: STXH WHERE tdoobject='EKPO' AND tdname BETWEEN ebeln+'00000' AND ebeln+'99999'
```

Step 3: For each STXH row — read content via READ\_TEXT

```
CALL FUNCTION 'READ_TEXT'

EXPORTING id=tdid language=tdspras name=tdname object=tdobject

TABLES lines=lt_lines.
```

Step 4: Convert ITF → plain text string

```
CALL FUNCTION 'CONVERT_ITF_TO_STREAM_TEXT'
```

```
TABLES i_text=lt_lines IMPORTING e_text=lv_content.
```

Step 5: DELETE existing from EKKT\_TEXT (restartability)

```
DELETE FROM ekkt_text WHERE ebeln=... AND text_id=... AND language=...
```

Step 6: INSERT into EKKT\_TEXT

```
INSERT ekkt_text: ebeln, ebelp, text_object, text_id, language,  
                 text_content, ref_text_obj, ref_text_id, ref_text_name,  
                 auth_ekorg, auth_bukrs, auth_bsart, created_by, created_at.
```

Step 7: UPDATE EKKO SDM\_VERSION = target (marks PO as migrated)

```
UPDATE ekko SET sdm_version = target WHERE ebeln = ...
```

Old STXH/STXL records NOT deleted – safety net for restart.

## 1 Create SDM class in SE24

Class name: ZCL\_SDM\_EKKO\_TEXT\_MIGRATION Inherits from: CL\_SDM\_PACKAGE\_MIGRATION Package: ZMMPOC  
Description: PO Notes SDM Migration Class

## 2 Implement all GET\_\* methods

These are simple one-liners returning constants: GET\_TABLE\_NAME → 'EKKO' GET\_SELECTIVE\_FIELD → 'EBELN'  
GET\_STATUS\_FIELD → 'SDM\_VERSION' GET\_PACKAGE\_SIZE → 30000 GET\_STATUS\_VALUE\_DONE →  
IF\_ZPO\_SDM\_CONSTANTS=>PO\_TEXT\_SDM\_VERSION

## 3 Implement MIGRATE\_DATA

Follow logic above — SELECT EKKO → read STXH → READ\_TEXT → CONVERT → DELETE+INSERT EKKT\_TEXT →  
UPDATE EKKO

## 4 Implement MIGRATE\_DATA\_FINISHED

SELECT COUNT(\*) FROM ekko WHERE sdm\_version < target If = 0 → migration complete If > 0 → raise error with count of  
remaining records

## 5 Activate class (Ctrl+F3)

All methods must be active before SDM can be executed

## 6 Test in DDCI sandbox — run SDM for test POs

Execute via SDM cockpit or direct ABAP call Expected: 3.5 hours for 13M records based on Sales performance test Package size  
30,000 stays within Open SQL limit of 32,767

■ Deliverable: ZCL\_SDM\_EKKO\_TEXT\_MIGRATION Active and tested on DDCI.

## 7

## POC — Write to Both STXH and EKKT\_TEXT via Modernized OData V4 API

*Prove the end-to-end flow works in DDCI before committing 2708 effort*

WHY POC: Without this proof, 2708 delivery could discover blockers mid-sprint. The POC runs in DDCI — completely separate from 2702 development. If POC works → cross-port to main development for 2708. Effort is NOT wasted — proven code can be reused directly.

## POC Goal Statement (as agreed in CE2702/CE2708 meeting)

Prove that when a PO is created via the new modernized PO OData V4 API: 1. Notes can be written to EKKT\_TEXT (new persistency) 2. Notes are ALSO written to STXH/STXL (old — dual-write) 3. CDS view on EKKT\_TEXT returns the text content correctly 4. Legacy transaction ME23N still shows the text (reads from STXH)

## 1 Set up DDCI system — spin off from current Maple/MEPO state

Do NOT use the main 2702 development system Spin off DDCI from current state of Maple (modernized PO) This isolates POC work — no risk to 2702 delivery

## 2 Model Notes in PO OData V4 API (coordinate with MEPO/Maple team)

Option A: Use Notes Reuse RAP component (if compatible with EKKT\_TEXT — from Activity 1) Option B: Model own text entity in PO RAP Business Object Minimum for POC: a simple CREATE action that accepts text content + text ID

## 3 Wire dual-write in OData V4 save flow

When PO saved via new API: Call SAVE\_TEXT → handler class routes to EKKT\_TEXT (Activity 4 registered) IS\_MIGRATION\_FINISHED=FALSE → BOTH EKKT\_TEXT and STXH written

## 4 Build basic CDS view on EKKT\_TEXT

CDS view: ZI\_POText Basic view on EKKT\_TEXT with typed EBELN+EBELP keys DCL for authorization using AUTH\_EKORG, AUTH\_BUKRS Analytics annotation for BW extraction (full C1 release in 2708)

## 5 Run end-to-end verification

Create PO via new OData V4 API with text → Verify: ✓ SE16N → EKKT\_TEXT: row with plain TEXT\_CONTENT, SOURCE equivalent ✓ SE16N → STXH: row also exists (dual-write working) ✓ SELECT from CDS view ZI\_POText: text content visible ✓ ME23N (legacy): text still visible (reads from STXH — IS\_MIGRATION\_FINISHED=FALSE)

## 6 Document POC results

Write POC findings document: What worked, what did not, blockers found Performance data from DDCI Recommendations for 2708 full implementation

## 7 Cross-port to main development for 2708

If POC passes → transport proven code to main development landscape 2708 starts from proven base — not from scratch

■ Deliverable: POC documented, DDCI proof available, cross-port plan confirmed.

## CE2702 Complete Summary



All 7 activities — why, what and deliverable

| # | Activity            | Why Required                             | Key Steps                                                   | Deliverable                        |
|---|---------------------|------------------------------------------|-------------------------------------------------------------|------------------------------------|
| 1 | Clarify Notes Reuse | Blocks table design decision             | 3 team syncs → Architecture Decision Record                 | ADR signed off                     |
| 2 | Design EKKT_TEXT    | Handler cannot be coded without table    | SE11 create → 17 fields → index                             | EKKT_TEXT Active                   |
| 3 | Handler Class       | STXH always used without it              | SE24 → inherit CL_RSTXT_PERSISTENCE_FRAMEWORK → 6 methods   | ZCL_PO_RSTXT_PERSISTENCE Active    |
| 4 | TTXOB Registration  | Handler never called without it          | TTXOB EKKO+EKPO → HANDLER_CLASS = class name                | Routing active, dual-write proven  |
| 5 | SDM_VERSION to EKKO | SDM tracks progress via this field       | Append Structure → SE11 → version constant                  | SDM_VERSION field in EKKO          |
| 6 | SDM Class           | Historic texts never migrated without it | SE24 → inherit CL_SDM_PACKAGE_MIGRATION → 7 methods         | ZCL_SDM_EKKO_TEXT_MIGRATION Active |
| 7 | POC in DDCI         | Blocks unknown 2708 delivery risks       | DDCI setup → model notes in API → dual-write → CDS → verify | POC proven, cross-port plan ready  |

### Key Technical Objects Created in CE2702

| Object                      | Type              | Transaction | Purpose                                    |
|-----------------------------|-------------------|-------------|--------------------------------------------|
| EKKT_TEXT                   | Database Table    | SE11        | New PO text persistency table              |
| ZCL_PO_RSTXT_PERSISTENCE    | ABAP Class        | SE24        | Handler class — routes SAVE_TEXT/READ_TEXT |
| ZEKKO_TEXT_SDM              | Append Structure  | SE11        | Adds SDM_VERSION to EKKO                   |
| IF_ZPO_SDM_CONSTANTS        | Interface         | SE24        | Version constants for SDM                  |
| ZCL_SDM_EKKO_TEXT_MIGRATION | ABAP Class        | SE24        | SDM migration class                        |
| ZI_POText                   | CDS View          | ADT         | Basic CDS view on EKKT_TEXT                |
| TTXOB (updated)             | Customizing Table | SE11        | Handler class registered for EKKO/EKPO     |